HOW-TO GUIDE: Using OpenCV + GStreamer to Capture USB Camera Video & Perform Motion Detection on Beagley-AI

Introduction:

Many online tutorials explain how to use OpenCV or how to use GStreamer pipelines, but they typically assume:

- You are using simple V4L2 access (`VideoCapture(0)`)
- You are not working on an embedded board with limited compute and no display

For the ENSC 351 project, we needed a headless, stable, low-CPU video pipeline that works reliably on the BeagleY-AI and supports real-time motion detection.

Hardware

- BeagleY-AI or similar Linux SBC
- USB camera (UVC recommended)

Software

On the beagle:

1. Install OpenCV and GStreamer plugins:

```
sudo apt install libopencv-dev
```

```
sudo apt install -y \ gstreamer1.0-tools \ gstreamer1.0-plugins-base \ gstreamer1.0-plugins-good \ gstreamer1.0-plugins-bad \ gstreamer1.0-plugins-ugly \ gstreamer1.0-libav
```

2. Check the device nodes:

```
ls /dev/video*
```

Expected output (example):

```
tyler@BeagleBone:~$ ls /dev/video*
/dev/video-usb-cam0  /dev/video0  /dev/video2  /dev/video4
/dev/video-usb-cam1  /dev/video1  /dev/video3
```

This command checks what video format the camera supports

```
v4l2-ctl --device=/dev/video1 --all
```

Expected output:

```
tyler@BeagleBone:~$ v4l2-ctl --device=/dev/video3 --all
Driver Info:
        Driver name      : uvcvideo
        Card type        : USB Webcam gadget: UVC HD Camer
        Bus info         : usb-xhci-hcd.5.auto-1.2
        Driver version   : 6.6.58
        Capabilities     : 0x84a00001
                Video Capture
                Metadata Capture
                Streaming
                Extended Pix Format
                Device Capabilities
        Device Caps      : 0x04200001
                Video Capture
                Streaming
                Extended Pix Format
```

```
Priority: 2
Video input : 0 (Camera 1: ok)
Format Video Capture:
        Width/Height      : 640/480
        Pixel Format      : 'MJPG' (Motion-JPEG)
        Field             : None
        Bytes per Line    : 0
        Size Image        : 102400
        Colorspace        : sRGB
        Transfer Function : Default (maps to sRGB)
        YCbCr/HSV Encoding: Default (maps to ITU-R 601)
        Quantization      : Default (maps to Full Range)
        Flags             :
Crop Capability Video Capture:
        Bounds      : Left 0, Top 0, Width 640, Height 480
        Default     : Left 0, Top 0, Width 640, Height 480
        Pixel Aspect: 1/1
```

This /dev/video1 node outputs MJPG

Step 2 — GStreamer pipeline captures the USB camera and streams MJPEG over TCP

To ensure a stable video source for OpenCV, we stream the camera through a local MJPEG TCP server using GStreamer. This avoids V4L2 driver issues on the BeagleY-AI and ensures that OpenCV receives clean, decoded frames.

Run the following pipeline on the Beagley-AI:

```
gst-launch-1.0 v4l2src device=/dev/video3 ! \

image/jpeg,width=640,height=480 ! jpegdec ! \

videoconvert ! jpegenc ! multipartmux ! \

tcpserversink host=0.0.0.0 port=8554
```

If the pipeline opens successfully, the BeagleY is now hosting a MJPEG video stream on localhost:8554, ready for OpenCV to use

```
tyler@BeagleBone:~$ gst-launch-1.0 -v v4l2src device=/dev/video3 do-timestamp=tr
ue !   image/jpeg !   jpegdec !   videorate ! video/x-raw,framerate=15/1 !   vid
eoconvert !   jpegenc !   queue max-size-buffers=1 leaky=downstream !   multipar
tmux boundary=frame !   tcpserversink host=0.0.0.0 port=8554
Setting pipeline to PAUSED ...
Pipeline is live and does not need PREROLL ...
/GstPipeline:pipeline0/GstTCPServerSink:tcpserversink0: current-port = 8554
Pipeline is PREROLLED ...
Setting pipeline to PLAYING ...
/GstPipeline:pipeline0/GstV4l2Src:v4l2src0: crop-bounds = < (int)0, (int)0, (int
)640, (int)480 >
New clock: GstSystemClock
```

```
ipped/left-flopped/right-flipped/right-flopped/half-aspect/mixed-mono
/GstPipeline:pipeline0/GstMultipartMux:multipartmux0.GstPad:src: caps = multipar
t/x-mixed-replace, boundary=(string)frame
/GstPipeline:pipeline0/GstTCPServerSink:tcpserversink0.GstPad:sink: caps = multi
part/x-mixed-replace, boundary=(string)frame
Redistribute latency...
0:00:01.3 / 99:99:99.
```

The screenshot displays the Beagley running the Gstreamer camera streaming pipeline, which captures /dev/video3, decodes MJPEGm, re-encodes it and streams it over TCP on port 8554.

Step 3 — Opening the MJPEG Stream from OpenCV

With the BeagleY-AI streaming MJPEG on port 8554, the motion detection program connects to this feed using a GStreamer client pipeline inside OpenCV. This allows OpenCV to receive fully decoded BGR frames directly through appsink.

In your C++ program, construct the pipeline as:

```
std::string pipeline =
    "tcpclientsrc host=127.0.0.1 port=8554 ! "
    "multipartdemux ! "
    "jpegdec ! "
    "videoconvert ! video/x-raw,format=BGR ! "
    "appsink";
```

Then open the stream using:

```
VideoCapture cam(pipeline, cv::CAP_GSTREAMER);
```

What OpenCV prints if successful:

```
Opening camera...
[ WARN:0@0.418] global cap_gstreamer.cpp:1777 open OpenCV | GStreamer warning: C
annot query video position: status=0, value=-1, duration=-1
Camera opened.
Actual size  : 1920 x 1080
Actual FPS   : 15
```

With a working BGR frame stream from appsink, we can now apply the motion detection algorithm described in Step 4

Step 4 — Motion Detection Algorithm

4.1 Convert to grayscale + blur

```
cvtColor(frame, gray, COLOR_BGR2GRAY);
GaussianBlur(gray, gray, Size(21, 21), 0);
```

4.2 Compute frame difference

```
absdiff(firstFrame, gray, frameDelta);
```

4.3 Threshold + contour detection

```
threshold(frameDelta, thresh, 25, 255, THRESH_BINARY);
dilate(thresh, thresh, Mat(), Point(-1,-1), 2);
findContours(thresh, contours, RETR_EXTERNAL, CHAIN_APPROX_SIMPLE);
```

4.4 Motion detected event

```
if (contourArea(c) > MIN_MOTION_AREA)
```

Output:

```
Starting motion detection (headless). Press Ctrl+C to quit.
[MOTION] Motion detected at 2960987 ms
[MOTION] Motion detected at 2961052 ms
[MOTION] Motion detected at 2961118 ms
[MOTION] Motion detected at 2961185 ms
[MOTION] Motion detected at 2961252 ms
[MOTION] Motion detected at 2961319 ms
[MOTION] Motion detected at 2961385 ms
[MOTION] Motion detected at 2961452 ms
[MOTION] Motion detected at 2961519 ms
[MOTION] Motion detected at 2961585 ms
[MOTION] Motion detected at 2961651 ms
[MOTION] Motion detected at 2961718 ms
[MOTION] Motion detected at 2961809 ms
[MOTION] Motion detected at 2961851 ms
[MOTION] Motion detected at 2961919 ms
^C
```

Troubleshooting

1. Camera fails to open (FAILED TO OPEN camera)

```
Cause: Wrong device node or invalid pipeline.
```

Fix:

```
ls /dev/video*
gst-launch-1.0 v4l2src device=/dev/video1 ! image/jpeg ! jpegdec !
videoconvert ! autovideosink
```

Use whichever /dev/videoN successfully displays video.

2. GStreamer error: not-negotiated or Failed to allocate memory

Cause: Pipeline requests MJPEG, but the selected /dev/videoN does *not* output MJPEG (e.g., /dev/video3 was HEVC).
Fix: Switch to a device that supports MJPEG (e.g., /dev/video1).

```
v4l2-ctl --device=/dev/videoN --all
```

Choose the node showing Pixel Format: MJPG.

3. No frames during warm-up

Cause: Camera slow to initialize, or wrong video node.
Fix:

- Test the device with GStreamer
- Try a different USB port
- Lower resolution if needed:

```
gst-launch-1.0 v4l2src device=/dev/video1 ! \
image/jpeg ! jpegdec ! \
video/x-raw,width=640,height=480 ! videoconvert ! autovideosink
```

4. Motion triggers constantly

Cause: Noise or small brightness changes.
Fix:

```
const int MIN_MOTION_AREA = 1500;   // increase
GaussianBlur(gray, gray, Size(31, 31), 0);
```

5. Motion never triggers

Cause: Detection threshold too high.
Fix:

```
const int MIN_MOTION_AREA = 400;
```

Also, ensure lighting is sufficient for contrast.

6. High CPU usage

Cause: Higher-resolution MJPEG decoding and large image processing per frame.
Fix: Reduce resolution in pipeline:

```
video/x-raw,width=640,height=480,framerate=15/1
```

or increase loop sleep:

```
this_thread::sleep_for(chrono::milliseconds(20));
```

7. Code compiles on VM but not on BeagleY

Cause: Missing OpenCV development package.
Fix:

```
sudo apt install libopencv-dev
```

8. Failed to run camera

Cause: code for GStreamer framerate and resolution is not the same as OpenCV
Fix: Make both of them the same framerate throughout the code and commands.

Conclusion

In this guide, we showed how to:

- Detect and verify a USB camera on the BeagleY-AI
- Build and test a GStreamer pipeline that decodes MJPEG frames
- Open that pipeline inside OpenCV using `cv::CAP_GSTREAMER`
- Implement a lightweight background-subtraction motion detection algorithm
- Troubleshoot common GStreamer and OpenCV issues on an embedded board

This setup provides a reusable template for any ENSC 351 project that needs headless, real-time video processing on BeagleY-AI.

Copyright Permission

Tyler Lee, Ariful Shayun, Jeffrey Luan, and Franco Obnamia grant the ENSC 351 instructor permission to post this How-To guide online for future students. We retain all copyright.